



Milan Jovanović



EF CONCURRENCY

PESSIMISTIC LOCKING



Uma maneira inteligente de implementar o bloqueio pessimista no EF Core.

5 minutos de leitura · 13 DE ABRIL DE 2024 · · Gerenciar histórico de leitura

O EF Core está muito lento? Descubra como inserir dados até 14 vezes mais rápido (reduzindo o tempo de carregamento em 94%). Aumente o desempenho com nosso método integrado ao EF Core: Inserção, atualização, exclusão e mesclagem em massa. Junte-se aos mais de



Os dias do Xamarin estão contados - você está pronto para migrar para o .NET MAUI? A decisão da Microsoft de encerrar o suporte ao Xamarin em 1º de maio de 2024 está fundamentada na evolução da tecnologia. O que vem a seguir? [Descubra aqui.](#)

Apoie esta newsletter →

Às vezes, especialmente em cenários de alto tráfego, é absolutamente necessário garantir que apenas um processo possa modificar um dado por vez.

Imagine que você está criando o sistema de venda de ingressos para um show extremamente popular. Os clientes estão comprando ingressos avidamente, e os últimos podem se esgotar simultaneamente. Se você não tomar cuidado, vários clientes podem achar que garantiram o último lugar, o que pode levar a overbooking e decepção!

O Entity Framework Core é uma ferramenta fantástica, mas não possui um mecanismo direto para bloqueio pessimista. [O bloqueio otimista](#) (usando versões) pode funcionar, mas em cenários de alta contenção, pode levar a muitas tentativas.

Então, como podemos resolver esse problema com o EF Core?

O cenário em mais detalhes



```
public async Task Handle(CreateOrderCommand request)
{
    await using DbTransaction transaction = await unitOfWork
        .BeginTransactionAsync();

    Customer customer = await customerRepository.GetAsync(request.CustomerId);

    Order order = Order.Create(customer);
    Cart cart = await cartService.GetAsync(customer.Id);

    foreach (CartItem cartItem in cart.Items)
    {
        // Uh-oh... what if two requests hit this at the same time?
        TicketType ticketType = await ticketTypeRepository.GetAsync(
            cartItem.TicketTypeId);

        ticketType.UpdateQuantity(cartItem.Quantity);

        order.AddItem(ticketType, cartItem.Quantity, cartItem.Price);
    }

    orderRepository.Insert(order);

    await unitOfWork.SaveChangesAsync();

    await transaction.CommitAsync();

    await cartService.ClearAsync(customer.Id);
}
```

O exemplo acima é artificial, mas deve ser suficiente para explicar o problema. Durante o processo de finalização da compra, verificamos o código de segurança **AvailableQuantity** de cada ingresso.

O que acontecerá se recebermos solicitações simultâneas tentando comprar o mesmo ingresso?

Na pior das hipóteses, acabaremos vendendo mais ingressos do que temos disponíveis. Solicitações simultâneas podem visualizar os ingressos disponíveis para venda e concluir a compra.



SQL puro para o resgate!

Como o EF Core não oferece bloqueio pessimista diretamente, vamos recorrer ao bom e velho SQL. Vamos substituir a `GetAsync` chamada para buscar o ticket por `GetWithLockAsync`.

Felizmente, o EF Core facilita isso com **consultas SQL diretas** :

```
public async Task<TicketType> GetWithLockAsync(Guid id)
{
    return await context
        .TicketTypes
        .FromSql(
            $"
            SELECT id, event_id, name, price, currency, quantity
            FROM ticketing.ticket_types
            WHERE id = {id}
            FOR UPDATE NOWAIT" // PostgreSQL: Lock or fail immediately
        ).SingleAsync();
}
```

Entendendo a magia:

- **FOR UPDATE NOWAIT** Este é o princípio fundamental do bloqueio pessimista no PostgreSQL. Ele instrui o banco de dados a "Acessar esta linha, bloqueá-la e, se ela já estiver bloqueada, gerar um erro imediatamente."
- **Tratamento de erros** : envolveríamos nossa `GetWithLockAsync` chamada em um `try-catch` bloco para lidar adequadamente com falhas de bloqueio, seja tentando novamente ou notificando o usuário.

Como o EF Core não oferece uma maneira nativa de adicionar dicas de consulta, precisamos escrever consultas SQL diretas. Podemos usar a **SELECT FOR UPDATE** instrução do PostgreSQL para obter um bloqueio em nível de linha nas linhas selecionadas. Quaisquer transações concorrentes serão bloqueadas até



Tipos de trava e quando usá-las

Para evitar que a operação fique aguardando outras transações para liberar linhas bloqueadas, você pode combiná-la `FOR UPDATE` com:

- `NO WAIT` - Reporta um erro se a linha não puder ser bloqueada, em vez de aguardar.
- `SKIP LOCKED` - Ignora quaisquer linhas selecionadas que não possam ser bloqueadas.

Ignorar linhas bloqueadas tem uma ressalva: você obterá resultados inconsistentes do banco de dados. No entanto, isso pode ser útil para evitar disputas de bloqueio quando vários consumidores acessam uma tabela em formato de fila. A implementação do [padrão Outbox](#) é um ótimo exemplo disso.

SQL Server : Você usaria a `WITH (UPDLOCK, READPAST)` dica de consulta para um efeito semelhante.

Bloqueio pessimista versus transações serializáveis

Transações serializáveis oferecem o mais alto nível de consistência de dados. Elas garantem que todas as transações sejam executadas como se tivessem ocorrido em uma ordem estritamente sequencial, mesmo que aconteçam simultaneamente. Isso elimina a possibilidade de anomalias como leituras sujas (visualização de dados não confirmados) ou leituras não repetíveis (dados alterados entre as leituras).



banco de dados bloqueia todos os dados que a transação possa acessar.

- Esses bloqueios são mantidos até que toda a transação seja confirmada ou revertida.
- Qualquer outra transação que tente acessar os dados bloqueados será bloqueada até que a primeira transação libere seus bloqueios.

Embora as transações serializáveis ofereçam o máximo isolamento, elas têm um custo significativo:

- **Sobrecarga de desempenho** : Bloquear um grande bloco de dados pode afetar severamente o desempenho, especialmente em cenários de alta concorrência.
- **Impasses** : Com tantos bloqueios acontecendo, o risco de impasses aumenta. Eles ocorrem quando duas ou mais transações estão aguardando bloqueios mantidos umas pelas outras, criando um impasse.

O bloqueio pessimista `SELECT FOR UPDATE` oferece uma abordagem mais direcionada para o isolamento de dados. Você bloqueia explicitamente as linhas específicas que precisa modificar. Outras transações que tentam acessar as linhas bloqueadas são bloqueadas até que o bloqueio seja liberado.

Ao bloquear apenas os dados necessários, o bloqueio pessimista evita a sobrecarga de desempenho associada ao bloqueio total. Como você está bloqueando menos recursos, as chances de impasses (deadlocks) são menores.

Quando usar cada abordagem

**Milan Jovanović**

altamente sensíveis, onde até mesmo a menor inconsistência é inaceitável. Exemplos incluem transações financeiras e atualizações de prontuários médicos.

- **Bloqueio pessimista** : Uma ótima opção para a maioria dos casos de uso, especialmente em aplicações de alto tráfego. Oferece forte consistência, mantendo um bom desempenho e reduzindo os riscos de deadlock.

Remover

Espero que esta exploração do bloqueio pessimista tenha sido útil. É uma ferramenta poderosa para se ter à disposição em cenários onde a consistência absoluta dos dados é fundamental.

Tanto **as transações serializáveis** quanto o bloqueio pessimista `SELECT FOR UPDATE` são excelentes opções para garantir a consistência dos dados. Ao escolher, considere o nível de isolamento necessário, o impacto potencial no desempenho e a probabilidade de impasses (deadlocks).

Por hoje é só. Continuem sendo incríveis e nos vemos na semana que vem.

Comentários

Faça login para deixar um comentário

[Iniciar sessão com o Google](#)[Faça login com o GitHub](#)



Quando você estiver pronto, existem 4 maneiras pelas quais posso te ajudar:

1. **Arquitetura Limpa Pragmática:** Junte-se a mais de 4.900 alunos neste curso abrangente que ensinará o sistema que utilizo para lançar aplicações prontas para produção usando Arquitetura Limpa. Aprenda a aplicar as melhores práticas da arquitetura de software moderna.
2. **Arquitetura Monolítica Modular:** Junte-se a mais de 2.800 engenheiros neste curso aprofundado que transformará a maneira como você constrói sistemas modernos. Você aprenderá as melhores práticas para aplicar a arquitetura monolítica modular em um cenário real.
3. **APIs REST pragmáticas:** Junte-se a mais de 1.800 alunos neste curso que ensinará como criar APIs REST prontas para produção usando os recursos mais recentes do ASP.NET Core e as melhores práticas. Ele inclui um aplicativo de interface do usuário totalmente funcional que integraremos à API REST.
4. **Comunidade Patreon:** Junte-se a uma comunidade com mais de 5.000 engenheiros e arquitetos de software. Você também terá acesso ao código-fonte que uso nos meus vídeos do YouTube, acesso antecipado a vídeos futuros e descontos exclusivos nos meus cursos.

**Milan Jovanović**

Junte-se a mais de 73.000 engenheiros que estão aprimorando suas habilidades todos os sábados de manhã.

[Join 70K+ Engineers](#)

© 2026 Milan Jovanovic Tech DOO

Contato

